Practical No-02

Name- Reeya Borikar

Roll No- 16

Registration No- 23030072

AIM- Develop GUI Based Simulation for 2D-transformations.

CODE-

```
function gui_2d_transform_interactive_v2
    % GUI for 2D Transformations: Translation, Euclidean, Similarity, Affine, and Projective

    % --- Constants and Styling ---
    C_BG = [0.94 0.94 0.94];    % Light Gray Background
    C_BLUE = [0 0.48 0.8];      % MATLAB Blue for main actions
    C_TEXT = [0.1 0.1 0.1];     % Dark Gray Text
    C_PANEL = [1 1 1];          % White Panel Background
    C_OBJECT = [0.8 0 0];       % Red color for the object
    C_ORIGINAL = [0.5 0.5 0.5]; % Gray color for original object trace

    % --- UI Figure Setup ---
    % Increased figure width to accommodate more controls
    fig = uifigure('Name','2D Geometric Transformations','Position',[100 100 850 650],...
                'Color', C_BG, 'Resize', 'off');

    % --- Variables ---
    % Define the initial object vertices (a square centered near the origin)
    initial_vertices = [-5, -5, 5, 5;   % X-coordinates
                        -5, 5, 5, -5;   % Y-coordinates
                         1, 1, 1, 1];   % Homogeneous coordinate (for transformation matrices)

    current_vertices = initial_vertices; % Vertices after transformations
```

```
    % --- UI Component Definitions ---

    % --- Control Panel (Left Side) ---
    panelControls = uipanel(fig, 'Title', 'Transformation Controls', ...
        'Position', [20 20 320 610], ... % Increased width for clarity
        'BackgroundColor', C_PANEL, ...
        'ForegroundColor', C_TEXT, ...
        'FontWeight', 'bold', 'FontSize', 12);

    % Reset Button
    uibutton(panelControls,'push','Text','Reset Object & Axes',...
        'Position',[40 540 240 40],... % <-- ADJUSTED DOWN
        'BackgroundColor', [0.85 0.85 0.85], ...
        'FontSize', 14, ...
        'ButtonPushedFcn',@(~,~) resetObject());

    % Separator
    uipanel(panelControls, 'Position', [10 528 300 2], 'BorderType', 'none', 'BackgroundColor', [0.7 0.7 0.7]); %
<-- ADJUSTED DOWN

    % --- Dynamic Transformation Panel (for parameters) ---
    uilabel(panelControls,'Text','Select Transformation Type:',...
        'Position',[20 496 280 22], 'FontSize', 12, 'FontWeight', 'bold'); % <-- ADJUSTED DOWN

    ddTransform = uidropdown(panelControls,...
        'Items', {'Translation', 'Euclidean (R+T)', 'Similarity (S+R+T)', 'Affine', 'Projective'},...
        'Position',[20 464 280 30], ... % <-- ADJUSTED DOWN
        'ValueChangedFcn', @(~,~) updateParamPanel(), ...
        'FontSize', 10);

    panelParams = uipanel(panelControls, 'Title', 'Parameters', ...
```

```
    'Position', [10 10 300 440], ... % <-- ADJUSTED DOWN
    'BackgroundColor', C_PANEL, ...
    'ForegroundColor', C_TEXT, ...
    'FontWeight', 'bold', 'FontSize', 12);

% Status label
lblStatus = uilabel(fig,'Text','Ready. Click Reset to start.',...
    'Position',[30 5 400 22], 'HorizontalAlignment', 'left', ...
    'FontSize', 10, 'FontColor', C_TEXT);

% --- Axes (Right Side) ---
axDisplay = uiaxes(fig,'Position',[360 20 470 610], 'Box','on', 'Color', 'w');
title(axDisplay,'2D Transformation Simulation','FontSize',14, 'FontWeight','bold', 'Color', C_TEXT);
grid(axDisplay, 'on');
xlabel(axDisplay, 'X-Axis');
ylabel(axDisplay, 'Y-Axis');

% Initial Axis Limits
xlim(axDisplay, [-50 50]);
ylim(axDisplay, [-50 50]);
daspect(axDisplay, [1 1 1]); % Ensure aspect ratio is 1:1

% Initialize parameter panel and object
updateParamPanel();
resetObject();

% --- DYNAMIC PARAMETER PANEL FUNCTION ---
function updateParamPanel()
    % Clear previous controls
    delete(findobj(panelParams.Children));
```

```
mode = ddTransform.Value;

% Global parameter definitions (used by multiple modes)
editDx = []; editDy = []; editAngle = []; editSx = []; editSy = [];
editShearX = []; editShearY = []; editP1 = []; editP2 = [];

switch mode
    case 'Translation'
        y_pos = 400;
        uilabel(panelParams, 'Text', 'Translation (dx, dy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'dx:', 'Position', [20 y_pos 30 22]);
        editDx = uieditfield(panelParams, 'numeric', 'Value', 10, 'Position', [50 y_pos 70 22], 'Tag', 'dx');
        uilabel(panelParams, 'Text', 'dy:', 'Position', [150 y_pos 30 22]);
        editDy = uieditfield(panelParams, 'numeric', 'Value', 5, 'Position', [180 y_pos 70 22], 'Tag', 'dy');
        y_pos = y_pos - 60;

    case 'Euclidean (R+T)'
        y_pos = 400;
        uilabel(panelParams, 'Text', 'Translation (dx, dy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'dx:', 'Position', [20 y_pos 30 22]);
        editDx = uieditfield(panelParams, 'numeric', 'Value', 10, 'Position', [50 y_pos 70 22], 'Tag', 'dx');
        uilabel(panelParams, 'Text', 'dy:', 'Position', [150 y_pos 30 22]);
        editDy = uieditfield(panelParams, 'numeric', 'Value', 5, 'Position', [180 y_pos 70 22], 'Tag', 'dy');

        y_pos = y_pos - 50;
        uilabel(panelParams, 'Text', 'Rotation (Angle in Degrees, CCW):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'Angle:', 'Position', [20 y_pos 50 22]);
```

```
        editAngle = uieditfield(panelParams, 'numeric', 'Value', 30, 'Position', [70 y_pos 180 22], 'Tag', 'angle');
        y_pos = y_pos - 60;

    case 'Similarity (S+R+T)'
        y_pos = 400;
        uilabel(panelParams, 'Text', 'Scaling (sx, sy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'sx (Uniform Scale):', 'Position', [20 y_pos 120 22]);
        editSx = uieditfield(panelParams, 'numeric', 'Value', 1.5, 'Position', [160 y_pos 70 22], 'Tag', 'sx');

        y_pos = y_pos - 50;
        uilabel(panelParams, 'Text', 'Rotation (Angle in Degrees, CCW):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'Angle:', 'Position', [20 y_pos 50 22]);
        editAngle = uieditfield(panelParams, 'numeric', 'Value', 30, 'Position', [70 y_pos 180 22], 'Tag', 'angle');
        y_pos = y_pos - 50;
        uilabel(panelParams, 'Text', 'Translation (dx, dy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'dx:', 'Position', [20 y_pos 30 22]);
        editDx = uieditfield(panelParams, 'numeric', 'Value', 10, 'Position', [50 y_pos 70 22], 'Tag', 'dx');
        uilabel(panelParams, 'Text', 'dy:', 'Position', [150 y_pos 30 22]);
        editDy = uieditfield(panelParams, 'numeric', 'Value', 5, 'Position', [180 y_pos 70 22], 'Tag', 'dy');
        y_pos = y_pos - 60;
    case 'Affine'
        y_pos = 400;
        uilabel(panelParams, 'Text', 'Scaling (sx, sy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'sx:', 'Position', [20 y_pos 30 22]);
        editSx = uieditfield(panelParams, 'numeric', 'Value', 1.5, 'Position', [50 y_pos 70 22], 'Tag', 'sx');
```

```
            uilabel(panelParams, 'Text', 'sy:', 'Position', [150 y_pos 30 22]);
            editSy = uieditfield(panelParams, 'numeric', 'Value', 0.8, 'Position', [180 y_pos 70 22], 'Tag', 'sy');

            y_pos = y_pos - 50;
            uilabel(panelParams, 'Text', 'Shear (shx, shy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
            y_pos = y_pos - 30;
            uilabel(panelParams, 'Text', 'shx:', 'Position', [20 y_pos 40 22]);
            editShearX = uieditfield(panelParams, 'numeric', 'Value', 0.3, 'Position', [70 y_pos 70 22], 'Tag', 'shx');
            uilabel(panelParams, 'Text', 'shy:', 'Position', [150 y_pos 40 22]);
            editShearY = uieditfield(panelParams, 'numeric', 'Value', 0, 'Position', [200 y_pos 70 22], 'Tag', 'shy');

            y_pos = y_pos - 50;
            uilabel(panelParams, 'Text', 'Rotation (Angle in Degrees, CCW):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
            y_pos = y_pos - 30;
            uilabel(panelParams, 'Text', 'Angle:', 'Position', [20 y_pos 50 22]);
            editAngle = uieditfield(panelParams, 'numeric', 'Value', 15, 'Position', [70 y_pos 180 22], 'Tag', 'angle');
            y_pos = y_pos - 50;
            uilabel(panelParams, 'Text', 'Translation (dx, dy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
            y_pos = y_pos - 30;
            uilabel(panelParams, 'Text', 'dx:', 'Position', [20 y_pos 30 22]);
            editDx = uieditfield(panelParams, 'numeric', 'Value', 10, 'Position', [50 y_pos 70 22], 'Tag', 'dx');
            uilabel(panelParams, 'Text', 'dy:', 'Position', [150 y_pos 30 22]);
            editDy = uieditfield(panelParams, 'numeric', 'Value', 5, 'Position', [180 y_pos 70 22], 'Tag', 'dy');
            y_pos = y_pos - 60;
        case 'Projective'
            y_pos = 400;
            uilabel(panelParams, 'Text', 'Projective (Perspective Factors p1, p2):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
            y_pos = y_pos - 30;
            uilabel(panelParams, 'Text', 'p1:', 'Position', [20 y_pos 30 22]);
```

```
        editP1 = uieditfield(panelParams, 'numeric', 'Value', 0.005, 'Position', [50 y_pos 70 22], 'Tag', 'p1');
        uilabel(panelParams, 'Text', 'p2:', 'Position', [150 y_pos 30 22]);
        editP2 = uieditfield(panelParams, 'numeric', 'Value', 0.002, 'Position', [180 y_pos 70 22], 'Tag', 'p2');

        % Note: Projective transformations contain Affine, but for simplicity in a demo,
        % we often focus on the perspective components p1 and p2.
        % We'll use a simplified Affine matrix with T/R/S, then add p1/p2.
        y_pos = y_pos - 50;
        uilabel(panelParams, 'Text', 'Translation (dx, dy):', 'Position', [20 y_pos 260 22], 'FontWeight', 'bold');
        y_pos = y_pos - 30;
        uilabel(panelParams, 'Text', 'dx:', 'Position', [20 y_pos 30 22]);
        editDx = uieditfield(panelParams, 'numeric', 'Value', 10, 'Position', [50 y_pos 70 22], 'Tag', 'dx');
        uilabel(panelParams, 'Text', 'dy:', 'Position', [150 y_pos 30 22]);
        editDy = uieditfield(panelParams, 'numeric', 'Value', 5, 'Position', [180 y_pos 70 22], 'Tag', 'dy');
        y_pos = y_pos - 60;
    end

    % Add Apply button at the bottom
    uibutton(panelParams,'push','Text',['Apply ' mode],'Position',[50 10 200 40], ...
        'ButtonPushedFcn',@(~,~) applyTransformation(mode), ...
        'BackgroundColor', C_BLUE, 'FontColor', [1 1 1], 'FontSize', 12, 'FontWeight', 'bold');
end

% --- CORE TRANSFORMATION FUNCTIONS ---

function resetObject()
    current_vertices = initial_vertices;
    cla(axDisplay);
    hold(axDisplay, 'on');

    plotObject(axDisplay, initial_vertices, C_ORIGINAL, 1, ':');
```

```
    plotObject(axDisplay, current_vertices, C_OBJECT, 3, '-');

    hold(axDisplay, 'off');
    xlim(axDisplay, [-50 50]);
    ylim(axDisplay, [-50 50]);
    daspect(axDisplay, [1 1 1]);
    lblStatus.Text = 'Object reset. Original object shown in gray dashed lines.';
    title(axDisplay, '2D Transformation Simulation (Reset)');
end

function applyTransformation(type)
    if isempty(current_vertices)
        lblStatus.Text = 'Please click Reset Object first.';
        return;
    end
    % Initialize identity matrix (3x3 homogeneous transformation)
    H = eye(3);

    % Helper function to safely read a parameter
    function val = getParam(tag, default)
        obj = findobj(panelParams.Children, 'Tag', tag);
        if isempty(obj)
            val = default;
        else
            val = obj.Value;
        end
    end
    try
        switch type
            case 'Translation'
                dx = getParam('dx', 0); dy = getParam('dy', 0);
```

```
    H = [1 0 dx; 0 1 dy; 0 0 1];
    status_msg = sprintf('Translation applied: dx=%.1f, dy=%.1f', dx, dy);
case 'Euclidean (R+T)'
    dx = getParam('dx', 0); dy = getParam('dy', 0);
    angle_deg = getParam('angle', 0); angle_rad = deg2rad(angle_deg);
    c = cos(angle_rad); s = sin(angle_rad);
    R = [c -s 0; s c 0; 0 0 1];
    T = [1 0 dx; 0 1 dy; 0 0 1];
    H = T * R; % Order matters: Rotate, then Translate
    status_msg = sprintf('Euclidean applied (R+T): Angle=%.1f, dx=%.1f', angle_deg, dx);

case 'Similarity (S+R+T)'
    sx = getParam('sx', 1);
    dx = getParam('dx', 0); dy = getParam('dy', 0);
    angle_deg = getParam('angle', 0); angle_rad = deg2rad(angle_deg);
    c = cos(angle_rad); s = sin(angle_rad);
    % Uniform Scaling
    S = [sx 0 0; 0 sx 0; 0 0 1];
    R = [c -s 0; s c 0; 0 0 1];
    T = [1 0 dx; 0 1 dy; 0 0 1];
    H = T * R * S; % Order: Scale, then Rotate, then Translate
    status_msg = sprintf('Similarity applied (S+R+T): Scale=%.1f, Angle=%.1f', sx, angle_deg);
case 'Affine'
    sx = getParam('sx', 1); sy = getParam('sy', 1);
    shx = getParam('shx', 0); shy = getParam('shy', 0);
    angle_deg = getParam('angle', 0); angle_rad = deg2rad(angle_deg);
    dx = getParam('dx', 0); dy = getParam('dy', 0);
    c = cos(angle_rad); s = sin(angle_rad);

    % Full Affine Matrix (combining rotation, scaling, shearing)
    a11 = sx*c + shx*s; a12 = -sx*s + shx*c; a13 = dx;
```

```
        a21 = shy*c + sy*s; a22 = -shy*s + sy*c; a23 = dy;
        H = [a11 a12 a13;
             a21 a22 a23;
             0 0 1];
        status_msg = 'Affine applied (non-singular)';

    case 'Projective'
        p1 = getParam('p1', 0); p2 = getParam('p2', 0);
        dx = getParam('dx', 0); dy = getParam('dy', 0);

        % Simplified Projective Matrix (Projective part + Translation part)
        H = [1 0 dx;
             0 1 dy;
             p1 p2 1];
        status_msg = sprintf('Projective applied: p1=%.4f, p2=%.4f', p1, p2);
    otherwise
        status_msg = 'Unknown transformation type.';
        return;
end
% Apply the transformation matrix H to the current vertices
current_vertices = H * current_vertices;

% For projective transformation, normalize the homogeneous coordinates
if strcmp(type, 'Projective')
    w = current_vertices(3, :);
    current_vertices(1, :) = current_vertices(1, :) ./ w;
    current_vertices(2, :) = current_vertices(2, :) ./ w;
    current_vertices(3, :) = 1; % Reset homogeneous coordinate
end
updatePlot(type);
lblStatus.Text = status_msg;
```

```
    catch ME
        lblStatus.Text = ['Error during transformation: ', ME.message];
    end
end

% --- PLOTTING FUNCTIONS ---
function updatePlot(type)
    cla(axDisplay);
    hold(axDisplay, 'on');

    plotObject(axDisplay, initial_vertices, C_ORIGINAL, 1, ':');
    plotObject(axDisplay, current_vertices, C_OBJECT, 3, '-');

    all_x = [initial_vertices(1,:), current_vertices(1,:)];
    all_y = [initial_vertices(2,:), current_vertices(2,:)];

    padding = 10;
    xlim(axDisplay, [min(all_x)-padding, max(all_x)+padding]);
    ylim(axDisplay, [min(all_y)-padding, max(all_y)+padding]);

    title(axDisplay, ['2D Transformation Simulation (Last: ' upper(type) ')']);
    hold(axDisplay, 'off');
end

function plotObject(ax, vertices, color, lineWidth, lineStyle)
    % Extracts 2D coordinates and plots the closed polygon
    x = vertices(1, :);
    y = vertices(2, :);

    % Close the polygon by adding the first point again
```

```
        x_closed = [x, x(1)];
        y_closed = [y, y(1)];

        plot(ax, x_closed, y_closed, 'Color', color, 'LineWidth', lineWidth, 'LineStyle', lineStyle);

        % Mark the vertices
        scatter(ax, x, y, 40, color, 'filled');

        grid(axDisplay, 'on');
        daspect(axDisplay, [1 1 1]);
    end
end
```

OUTPUT-

2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Translation

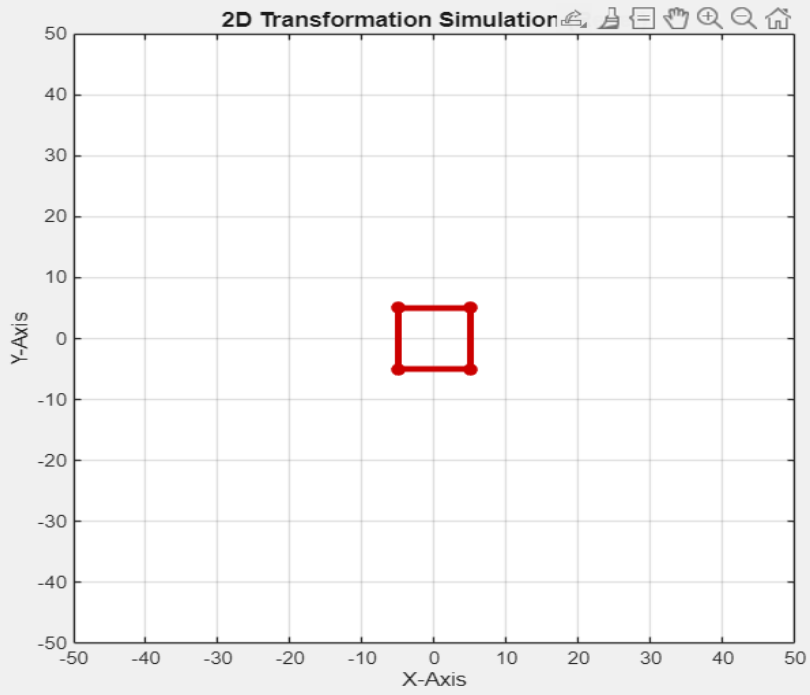
Parameters

Translation (dx, dy):

dx: 10 dy: 5

Apply Translation

Object reset. Original object shown in gray dashed lines.



2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Translation

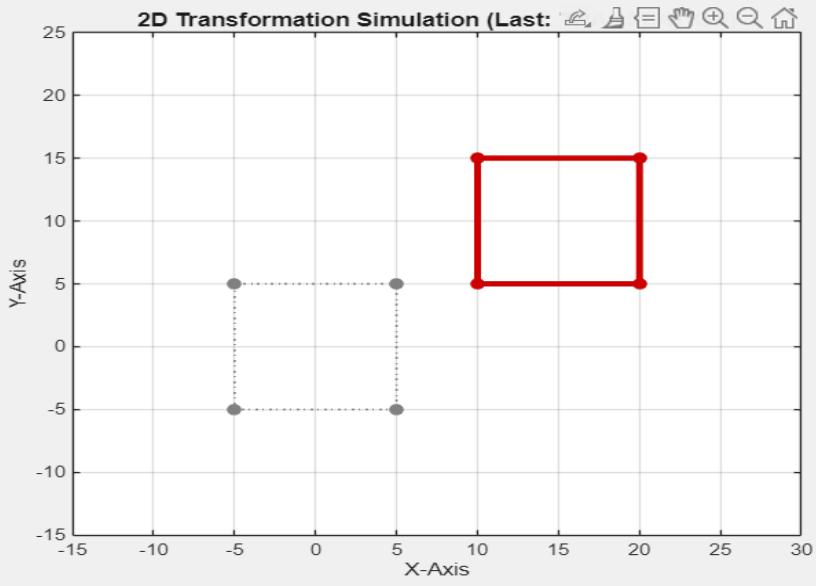
Parameters

Translation (dx, dy):

dx: 15 dy: 10

Apply Translation

Translation applied: dx=15.0, dy=10.0



2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Euclidean (R+T)

Parameters

Translation (dx, dy):

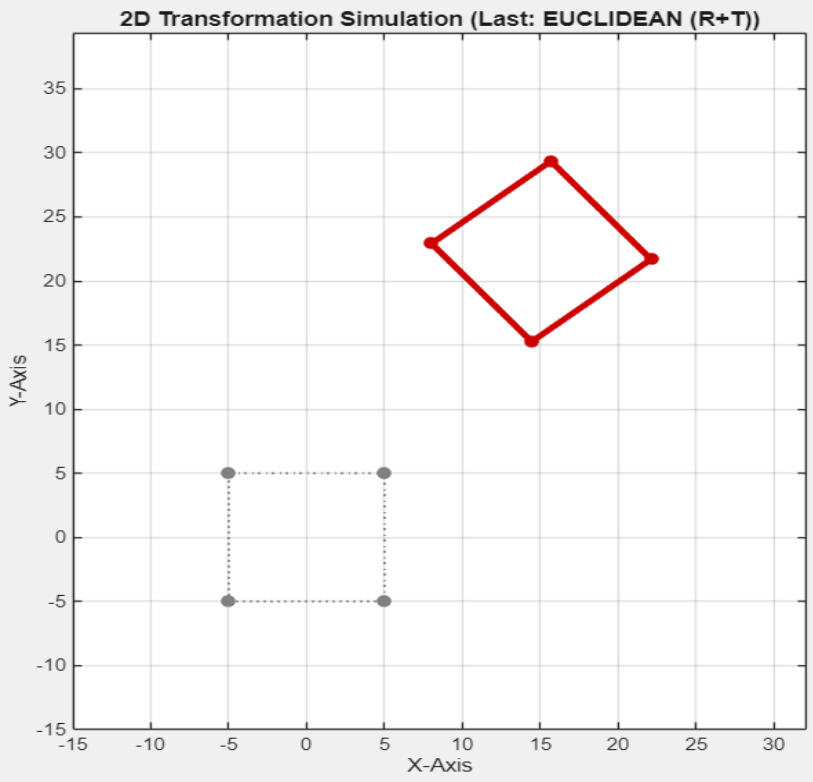
dx: 10 dy: 5

Rotation (Angle in Degrees, CCW):

Angle: 40

Apply Euclidean (R+T)

Euclidean applied (R+T): Angle=40.0, dx=10.0



2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Similarity (S+R+T)

Parameters

Scaling (sx, sy):

sx (Uniform Scale): 2

Rotation (Angle in Degrees, CCW):

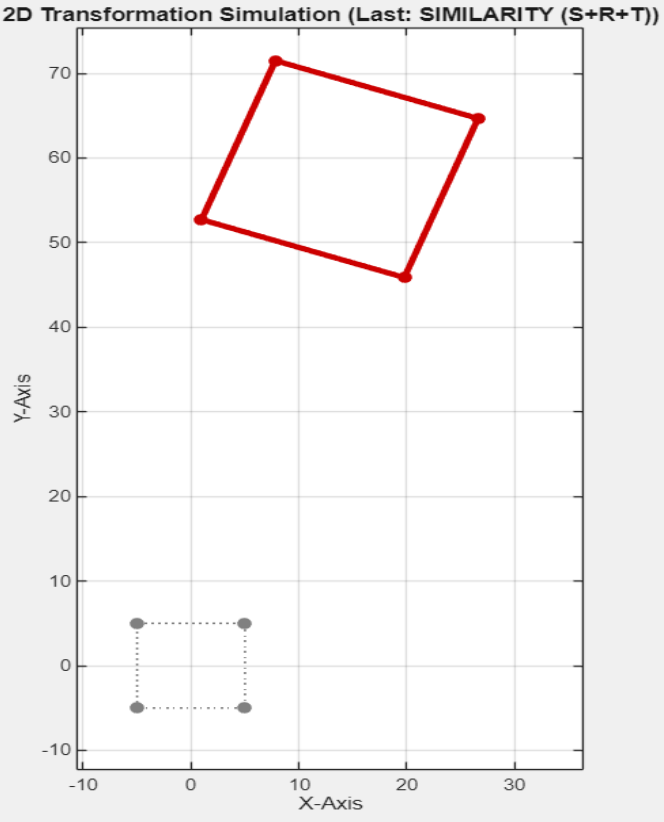
Angle: 30

Translation (dx, dy):

dx: 10 dy: 5

Apply Similarity (S+R+T)

Similarity applied (S+R+T): Scale=2.0, Angle=30.0



2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Affine

Parameters

Scaling (sx, sy):

sx: 1.5 sy: 0.8

Shear (shx, shy):

shx: 0.5 shy: 1

Rotation (Angle in Degrees, CCW):

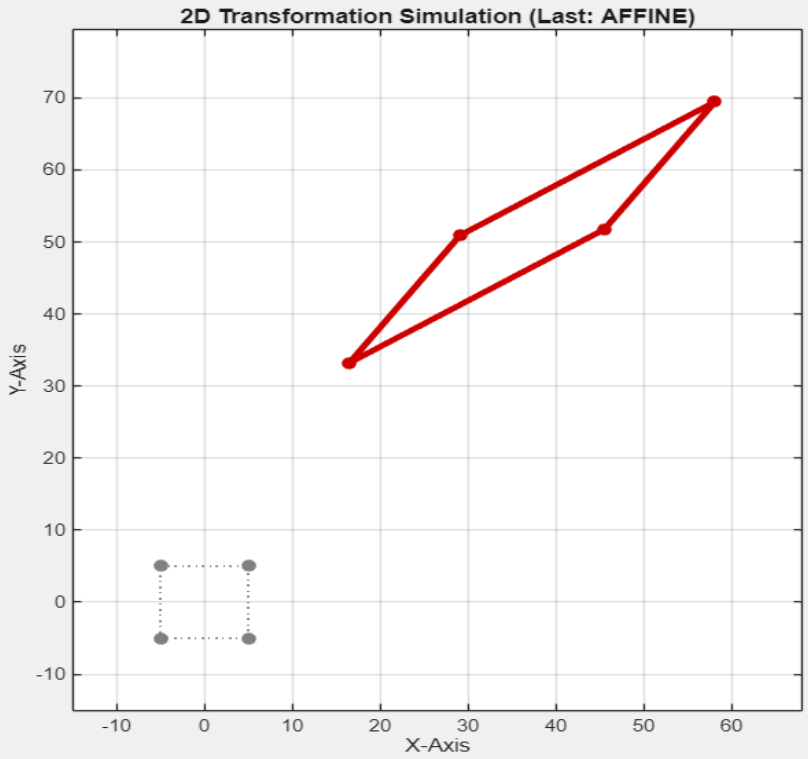
Angle: 15

Translation (dx, dy):

dx: 10 dy: 5

Apply Affine

Affine applied (non-singular)



2D Geometric Transformations

Transformation Controls

Reset Object & Axes

Select Transformation Type:

Projective

Parameters

Projective (Perspective Factors p1, p2):

p1: 0.015 p2: 0.02

Translation (dx, dy):

dx: 15 dy: 10

Apply Projective

Projective applied: p1=0.0150, p2=0.0200